Software Requirements Specification

POWER VISION ENGINEERING | Business Management System (PVE-BMS)

Document ID	Version	Date	Status
PVE-SRS-001	v1.0 Baseline (Updated Tech Stack)	17 October 2026	Version 1.0 — Baseline Incorporating Client Responses & Updated Tech Stack

APPROVED SYSTEM TECHNOLOGY STACK & ARCHITECTURE (Updated)

- **Development Team:** One Student Developer (Full-Stack Engineer)
- **Frontend Layer:** Angular + TypeScript (Single Page Application / Responsive UI)
- **Backend Framework:** Java + Spring Boot (RESTful Web Services API Architecture)
- **Database & ORM:** PostgreSQL (Deployment Database Layer) via Spring Data JPA + Hibernate (ORM)
- **Data Access / Integration:** Spring Data JPA repositories, Hibernate ORM, and selective JDBC for optimized query execution
- **Security & Auth:** Spring Security + JSON Web Tokens (JWT) for stateless authentication & Role-Based Access Control (RBAC)
- **Testing & Quality:** JUnit 5 + Spring Boot Test framework for unit, service, and repository integration testing
- **Version Control & CI/CD:** Git, GitHub repository hosting, and automated CI/CD pipeline (GitHub Actions for build, test, and package release)

Document Control

Field	Value
Project Team	One Student Developer (Full-Stack Engineering Role)
Client	Power Vision Engineering (Owner-managed retail & repair business)
Product	Local Shop & Business Management System (PVE-BMS)
Owner / Approver	Power Vision Engineering Owner (Name and signature pending final review)
Prepared By	Solo Student Developer
Inputs & Sources	Original 5-page requirements; 2-page clarification; requirements register v0.1; baseline v0.3; client responses O-01 to O-10 (03 Nov 2026); updated single-developer technology stack specification

Decision Status	Client responses incorporated. Confirmed facts are marked C; team recommendations remain P; ambiguous answers remain O. Architecture & tech stack updated for 1-developer delivery.
Revision History	v0.4: Consolidated SRS draft. v0.5: Incorporated client responses O-01 to O-10. v1.0: Baseline SRS with refined stack (Angular, Spring Boot, PostgreSQL, JWT, CI/CD, JUnit) for single developer.

Evidence & Requirement Status Notation

Mark	Meaning & Operational Context
C — Client-stated	Directly present in client notes or subsequent clarification; detailed implementation details may still be open.
P — Proposed	Developer recommendation only; requires owner agreement before committing to scope.
O — Open	Business rule / requirement not fully determined; must be resolved before relevant design or release.
D — Deferred	Out of Version 1 scope, subject to future contract or owner sign-off.

Priority: Must (Core Candidate), Should (Valuable Next), Could (Later). All priority assignments are provisional until final owner sign-off.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the intended behavior, system boundaries, architectural constraints, security provisions, and verifiable quality characteristics of the Power Vision Engineering Business Management System (PVE-BMS). It serves as the baseline for implementation, testing, client validation, and project evaluation.

1.2 Business Background

Power Vision Engineering operates an owner-managed retail and engineering business selling electrical/wiring products, power tools, and spare parts. The business also performs power-tool repairs, tool distribution, and site engineering projects. Operating divisions include Retail, Services, Distribution (DS), and Engineering. The business manages approximately 100 suppliers, around 10 permanent staff members, plus external site workers. [C]

1.3 Problem Statement

The owner currently handles shop operations manually, leading to time-consuming product/stock entry, billing bottlenecks, difficulty locating stock across multiple physical racks, lack of automated low-stock alerts, and inadequate financial/sales reporting. Additional business challenges include manual credit tracking for selected

commercial clients, margin protection, intake tracking for power tool repairs, and managing 3 shop computers sharing real-time inventory updates. [C]

1.4 Scope and Boundaries

• Proposed Version 1 (MVP Scope):

Locally accessible multi-user POS, product & category management, barcode lookup and printing, multi-rack stock tracking, supplier record management, stock receiving, Excel product import, credit customer ledger (30-day payment cycle), owner-controlled pricing/discounts, gross profit operational reports, role-based access control (JWT auth), basic power-tool repair intake with consented photo attachment, and weekly automated database backups. [P scope combining C needs]

• Out of Scope for Version 1 (Deferred):

Full repair service lifecycle management, manufacturing ERP, full distribution ERP, engineering project job costing, payroll/EPF management, biometric fingerprint attendance, automated PDF catalog extraction, and seasonal AI forecasting. [D; subject to future releases]

1.5 Technical Architecture & Single-Developer Delivery Baseline

The system is engineered for full implementation and maintenance by a single developer. The technical architecture guarantees clear separation of concerns, rapid feature delivery, high performance over the local shop network, and straightforward maintenance:

- **Frontend Architecture:** Modular Angular frontend with TypeScript components, services, and reactive forms for seamless user interaction.
- **Backend Architecture:** Layered REST API built with Java and Spring Boot, utilizing Spring Data JPA and Hibernate ORM for efficient relational data mapping to PostgreSQL.
- **Security Architecture:** Robust security implementation using Spring Security and JWT tokens, enforcing fine-grained Role-Based Access Control (RBAC) at the server API level.
- **Quality Assurance:** Comprehensive test suites using JUnit 5 and Spring Boot Test for critical domain logic (margin calculations, rack allocation, credit ledgers).
- **DevOps & CI/CD:** Git repository hosted on GitHub with an automated CI/CD pipeline executing build compilation, static analysis, unit test suites, and container/executable packaging.

1.6 Definitions and Acronyms

Term	Meaning & Context
POS	Point of Sale: transaction entry, product scanning, checkout, and receipt issuance
SKU	Stock Keeping Unit: unique internal product identifier
Gross Margin	Formula: (Net selling price - Assigned item cost) / Net selling price * 100 [C]; cost basis details still [O]
Rack Allocation	Automated mapping of product stock deductions to specific physical shop racks during checkout
Receivable	Outstanding monetary amount owed by a credit customer

RBAC	Role-Based Access Control: restricting API endpoints and UI views based on user roles
JWT	JSON Web Token: compact, URL-safe means of representing claims to be transferred between client and server
ORM	Object-Relational Mapping: technique (Hibernate/JPA) mapping Java domain objects to PostgreSQL tables
SRS	Software Requirements Specification
MVP	Minimum Viable Product: initial production release baseline

2. Stakeholders and Operating Context

The system supports multiple operational roles within the shop context. Roles are managed centrally via Spring Security permissions:

Actor	Business Responsibility / System Goal	Evidence Status
Owner / Manager	Review overall sales, set pricing, control discounts, review gross margins, manage suppliers, view restricted profit reports, approve invoice voids, manage system backups.	C
Cashier / Retail Staff	Perform customer product search, scan barcodes, issue sales invoices, process cash/card/bank/cheque payments, attach credit sales to approved customer profiles.	P role (derived from POS need)
Data-Entry / Stock Worker	Maintain product catalog, record received goods into physical racks, execute validated Excel imports, request stock adjustments.	C role distinction
Repair Intake Staff	Register customer details, capture power tool specifications, upload consented photos, record advance payments (25% for trusted clients), print repair receipts.	P role (C workflow)
Customer	Purchases products, receives permitted discounts, buys on 30-day credit (if authorized), submits tools for repair intake.	C
Supplier	Provides electrical/tool products and price data; approximately 100 active suppliers.	C
Administrator / Single Dev	System installation, DB schema migrations, backup configuration, GitHub CI/CD pipeline maintenance, user account provisioning.	P

2.1 Technical Environment & Hardware Interfaces

- • Shop LAN Environment: Three workstation PCs connected over a local area network (LAN). System must operate locally without requiring public internet access. [C]
- • Barcode & POS Hardware: Input via USB/Wireless barcode scanners; output via dedicated barcode label printer and receipt printer. [C]
- • Deployment Stack: Central PostgreSQL database instance hosted on the primary shop computer or dedicated mini-server; Spring Boot backend application server running locally; Angular web app served locally to shop browsers. [P]

3. Functional Requirements

Functional requirements are classified by subsystem. All backend logic must be backed by JUnit test suites and secured via Spring Security JWT middleware.

3.1 Identity, Access Control & Role Administration (AUTH)

ID / Status	Requirement Statement (System Shall)	Proposed Acceptance Evidence
AUTH-01 C/Must	The system shall require authenticated account login (JWT token authentication) before granting access to shop business features.	Unauthenticated REST API requests return 401 Unauthorized; Angular router blocks unauthenticated navigation.
AUTH-02 C/Must	The system shall restrict features based on staff role (Owner, Cashier, Data-Entry, Repair Intake).	Data-entry user attempting to modify selling price is blocked by UI and REST API.
AUTH-03 P/Must	The Spring Security backend shall enforce role permissions independently of UI controls.	Direct HTTP POST/PUT requests to protected endpoints without proper JWT role claims return 403 Forbidden.
AUTH-04 P/Should	The system shall log identity, timestamp, and operation details for sensitive business actions (price changes, manual stock adjustments, invoice voids).	Audit trail table records user ID, timestamp, entity affected, and change details.

3.2 Products, Barcodes, Suppliers & Bulk Data Import (PRD / SUP / IMP)

ID / Status	Requirement Statement (System Shall)	Proposed Acceptance Evidence

PRD-01 C/Must	The system shall maintain a searchable product catalog categorized by product line/range.	Product lookup by name, category, or range returns correct items in Angular UI.
PRD-02 C/Must	The system shall link products to their respective supplier records and allow supplier lookup.	User views supplier profile and sees list of associated products.
PRD-03 C/Must	The system shall support standard product barcodes and enable instant product selection via scanner input.	Scanning barcode populates POS cart item within < 200ms.
PRD-04 P/Should	The system shall generate internal SKUs and format printable barcode labels for products lacking supplier barcodes.	System generates unique SKU and renders printable barcode format.
SUP-01 C/Must	The system shall maintain profiles for approximately 100 suppliers with contact details and supplier IDs.	Search and edit supplier profiles without duplicate key errors.
IMP-01 C/Must	The system shall support bulk product import via Excel files (.xlsx/.csv), validating rows against product IDs and barcodes.	Excel file upload previews mapped columns and valid/invalid rows before committing.
IMP-02 P/Must	The import service shall flag duplicate IDs/barcodes and row errors. Existing database records shall not be silently overwritten.	Duplicate rows are highlighted in review UI; user explicitly approves updates.
IMP-03 C/D	The system should eventually extract product data from supplier PDF invoices.	Deferred out of Version 1 MVP.

3.3 Multi-Rack Inventory & Stock Management (INV / PUR)

ID / Status	Requirement Statement (System Shall)	Proposed Acceptance Evidence
INV-01 C/Must	The system shall track inventory quantities for a single product across multiple physical rack locations.	Product search displays stock breakdown (e.g., Rack A: 10, Rack B: 5, Total: 15).
INV-02 C/Must	Staff shall search for products and instantly view assigned cupboard/rack locations and available stock.	Search results display rack location codes

		alongside quantities.
INV-03 C/Must	The system shall automatically update rack stock balances upon completed sales, stock receipts, and adjustments.	Completing sale of 3 units reduces specific rack balance from 10 to 7.
INV-04 C/Must	The system shall trigger visual warnings when product stock falls below configured low-stock thresholds.	Low-stock alert dashboard lists items at or below threshold.
INV-05 C/Must	During POS checkout, the system shall allocate stock from configured racks automatically in a deterministic sequence.	Sale auto-allocates stock according to configured default rack order.
INV-06 P/Must	The system shall retain a complete stock movement log for all receipts, sales, adjustments, and returns.	Stock movement log displays timestamp, user, quantity delta, reason, and rack ID.
INV-07 C/Must	The system shall prevent sales when total available stock is insufficient, blocking negative inventory balances.	POS blocks checkout if quantity requested exceeds total available stock.
PUR-01 P/Must	Authorized users shall record incoming stock shipments specifying supplier, cost price, quantity, and destination rack.	Stock receiving transaction updates rack inventory and logs purchase movement.

3.4 Point of Sale, Pricing & Billing (POS)

ID / Status	Requirement Statement (System Shall)	Proposed Acceptance Evidence
POS-01 C/Must	Cashiers shall construct sales bills by searching products or scanning barcodes, selecting payment method (cash, card, bank transfer, cheque).	Multi-line cart creates saved invoice and sends print job to receipt printer.
POS-02 C/Must	The system shall allow cashiers to apply approved percentage or custom discounts for eligible regular customers.	Cashier enters permitted discount; invoice updates total balance correctly.
POS-03 C/Must (revised)	The system shall calculate selling prices and gross margin info without blocking sales below target margin (no automated floor block). Price overrides remain owner-restricted.	Cashier can complete sale below margin if needed; price editing restricted to owner.

POS-04 P/Must	Completing checkout shall commit invoice creation, payment entry, rack stock deduction, and ledger updates as an atomic Spring transaction.	Simulated database failure during checkout results in full transaction rollback.
POS-05 C/Should (revised)	The system shall allow only the owner role to void/correct saved invoices. Routine invoice reprinting is disabled unless re-authorized.	Non-owner attempt to void invoice fails with security exception; void action logged.
POS-06 P/Should	The checkout UI shall support rapid keyboard shortcuts and barcode scanner operations for fast cashier workflow.	Cashier completes full 3-item sale using keyboard/scanner in < 15 seconds.

3.5 Customer Credit Accounts (CRD)

ID / Status	Requirement Statement (System Shall)	Proposed Acceptance Evidence
CRD-01 C/Must	The system shall identify credit customers by Customer Name and Phone Number.	Cashier attaches credit sale by selecting verified customer profile.
CRD-02 C/Must	The system shall record credit invoices against owner-approved customer accounts with a standard 30-day payment term.	Credit sale updates customer outstanding account receivable balance.
CRD-03 P/Must	The system shall record partial or full debt payments independently of original invoice records.	Customer payment reduces total account balance; original invoice line history preserved.
CRD-04 C/O	Credit availability is restricted to owner-approved commercial clients. Overdue extra charge details remain subject to client approval.	System blocks credit checkout for non-approved customers; overdue charges unapplied pending rule confirmation.

3.6 Power-Tool Repair Intake (REP-INT)

ID / Status	Requirement Statement (System Shall)	Proposed Acceptance Evidence
REP-INT-01 C/Must	The system shall register repair intake records including unique repair ID, customer name, phone, tool model, issue	Saved repair record searchable by auto-generated repair reference

	description, estimated cost, and advance payment.	ID.
REP-INT-02 C/Must (revised)	The system shall allow attaching repair tool photos only after explicit customer permission is logged. Repair data and photos subject to 6-month retention.	Photo upload requires checking consent checkbox; photo stored securely with 6-month auto-archive flag.
REP-INT-03 C/Should (revised)	The system shall print a repair intake receipt and advance payment voucher upon intake confirmation.	Intake generates printable PDF receipt displaying tool details, estimate, and advance.
REP-INT-04 O	The system shall record a 25% advance for trusted customers. Refund and cancellation rules remain open pending owner definition.	Advance payment recorded against repair ticket; refund automation disabled.

3.7 Financial & Operational Reporting (RPT)

ID / Status	Requirement Statement (System Shall)	Proposed Acceptance Evidence
RPT-01 C/Must (revised)	The system shall generate daily, weekly, monthly, and annual Gross Profit reports accessible exclusively by the Owner.	Non-owner roles cannot view or request profit report APIs; reports exclude shop operational overheads.
RPT-02 C/Must	The system shall generate inventory valuation, daily sales summaries, and restock alert reports.	Owner filters reports by date range and exports results to Excel/PDF.
RPT-03 C/Should	The system shall provide comparative sales trends over selected timeframes once historical data is accumulated.	Trend dashboard displays sales comparison graphs across months/years.
RPT-04 P/Must	Financial reports shall explicitly label interim profit metrics as 'Gross Profit / Estimated Sales Summary' pending finalized cost/tax rules.	Report footers clearly display cost basis assumptions and exclusions.

4. Business Rules Summary

ID / Status	Business Rule Definition & Outstanding Clarifications

BR-01 C/O	Gross Margin Formula: Gross Margin % = (Net Selling Price - Assigned Cost) / Net Selling Price * 100. [C] Product cost assignment method (FIFO vs. Weighted Average) and 2% tax calculation remain open [O].
BR-02 C/O	Credit Authorization: Credit sales limited to owner-approved commercial accounts with 30-day payment terms. Duplicate customer names resolved via phone number key. Overdue charge rules open [O].
BR-03 C/O	Rack Inventory Allocation: Products occupy multiple racks. System automatically deducts stock during POS checkout. Automated rack picking order sequence requires owner confirmation [O].
BR-04 C/O	Barcode Management: Existing product barcodes scanned directly. Internal SKUs generated for unbarcoded items. Barcode printing formatted for standard label printers.
BR-05 C/O	Repair Intake Policy: 25% advance collected for trusted clients. Customer consent mandatory before photo upload. 6-month record retention enforced.
BR-06 C/O	Supplier Returns: Returns processed upon supplier vehicle arrival. Stock restoration requires authorized inspection before re-entering saleable rack inventory [P].
BR-07 C/O	Role Permissions: Data-entry staff manage catalog/stock; Cashiers process sales; Owner retains exclusive authority for price changes, discounts, invoice voids, and profit reports.

5. Role Permissions Matrix (Proposed)

The matrix below outlines the granular Role-Based Access Control (RBAC) enforced by Spring Security REST endpoints and Angular UI guards:

Activity / System Feature	Owner / Manager	Cashier Role	Data-Entry / Stock	Repair Intake Staff
View Product Catalog & Rack Stock	Allowed	Allowed	Allowed	Allowed (Read-only)
Create Sales & Receive Payments	Allowed	Allowed	Denied	Denied
Edit Product Prices & Discounts	Allowed	Denied	Denied	Denied
Receive Stock & Bulk Import	Allowed	Denied	Allowed	Denied
Adjust Rack Inventory Balances	Full Approval	Denied	Request Only	Denied
View Cost & Gross Profit Reports	Allowed	Denied	Denied	Denied

Register Power-Tool Repairs	Allowed	Denied	Denied	Allowed
Void/Correct Invoices	Allowed	Denied	Denied	Denied
System Administration & Backups	Admin Only	Denied	Denied	Denied

6. Primary Use Cases & End-to-End Flows

UC-01: Goods Receiving & Rack Allocation

- **Main Flow:** Stock Worker authenticates; opens Receiving module; selects Supplier and Product; inputs Unit Purchase Cost, Quantity, and Destination Rack; confirms receipt.
- **System Execution:** Spring Boot backend validates payload, updates PostgreSQL rack balance, creates GoodsReceipt and InventoryMovement records in an atomic transaction.
- **Traceability:** PUR-01, INV-01, INV-03, INV-06
- **Exceptions Tested:** Invalid quantity <=0, unlinked SKU, database lock timeout.

UC-02: POS Checkout & Automated Inventory Deduction

- **Main Flow:** Cashier scans product barcodes or searches catalog; modifies quantities; applies eligible customer discount; selects payment method (Cash/Card/Credit); clicks Complete Checkout.
- **System Execution:** Spring Security validates JWT token; backend verifies available stock across racks; auto-allocates rack deductions; records Invoice, Payments, and Stock Move; returns success payload.
- **Traceability:** POS-01..04, INV-05, CRD-01..03
- **Exceptions Tested:** Stock shortage (blocks checkout), unverified credit customer, transaction failure (triggers rollback).

UC-03: Bulk Excel Catalog Import

- **Main Flow:** Data-Entry worker uploads Excel catalog spreadsheet via Angular UI; system displays row preview with validation indicators; user reviews and clicks Import.
- **System Execution:** Spring Boot service parses file using Apache POI, validates barcode/SKU uniqueness, flags duplicate rows, inserts valid products without overwriting existing data.
- **Traceability:** IMP-01, IMP-02

7. System Architecture & Technical Specifications

7.1 Architectural Overview

The PVE-BMS follows a modern, decoupled 3-tier client-server architecture tailored for single-developer maintainability and high local LAN performance:

- **1. Presentation Layer (Frontend):** Single Page Application (SPA) built with Angular and TypeScript, featuring Angular Material components, reactive forms, and RxJS state management.
- **2. Application Layer (Backend):** RESTful API services built with Java 17 and Spring Boot framework. Employs Spring Security for JWT authentication/authorization and Spring Data JPA / Hibernate ORM for domain persistence.
- **3. Data Persistence Layer:** Relational PostgreSQL database instance running locally. Stores entity tables, inventory movement ledgers, spatial rack mappings, and audit logs with strict transactional integrity.

7.2 Conceptual Data Model (Entities)

Key domain entities mapped via Hibernate ORM include:

- **Authentication:** User, Role, Permission (JWT authentication & RBAC authorization)
- **Master Data:** Product, Category, Supplier, Barcode, StockLocation (Racks)
- **Inventory:** InventoryBalance (per-rack stock), InventoryMovement, GoodsReceipt
- **Sales & Billing:** Invoice, InvoiceLine, SaleAllocation, Payment, Customer, CreditLedgerEntry
- **Repairs & Audit:** RepairIntake, RepairPhoto (6-month retention tracking), AuditEvent

8. Non-Functional Requirements (NFRs)

ID	Quality Requirement & Verification Strategy	Status / Target
NFR-SEC-01	Passwords hashed using BCrypt. REST API protected via JWT tokens and Spring Security role evaluations. Verified via security integration tests.	P (Implemented)
NFR-DATA-01	Database transaction integrity: Sales checkout must complete atomically or roll back fully on failure. Verified via Spring @Transactional unit tests.	P (Mandatory)
NFR-CON-01	Multi-user LAN concurrency: System must support 3 simultaneous shop PCs without lost updates or stock race conditions. Verified via jMeter stress tests.	C Context / P Test
NFR-PERF-01	POS barcode scanning and cart calculation response time < 200ms over shop LAN. Verified on standard shop desktop hardware.	P Target
NFR-BACK-01	Weekly automated database backup routine to local/external media with backup integrity verification. Verified via restore test script.	C/Must (Revised)
NFR-PRIV-01	Customer phone numbers and repair photos restricted to authorized users with explicit customer consent logging and 6-month retention enforcement.	P/O
NFR-TEST-01	Backend business logic (margin calculations, stock allocation, credit balances) must maintain > 80% test coverage using JUnit 5.	P (Single Dev Standard)
NFR-CICD-01	GitHub Actions pipeline must automatically execute compile, linting, JUnit testing, and package build on every Git commit/PR.	P (DevOps Baseline)

9. Dependencies, Assumptions and Exclusions

- **Assumption A-01:** A dedicated PC or mini-server on the shop LAN will host the PostgreSQL database and Spring Boot backend. [P]
- **Assumption A-02:** Initial product list and opening stock balances will be formatted in Excel for bulk import prior to system go-live. [P]

- **Exclusion E-01:** Biometric attendance, EPF/ETF payroll, distribution ERP, and project accounting are explicitly excluded from Version 1. [D]
- **Exclusion E-02:** Automated AI PDF extraction and sales forecasting are deferred to post-v1 releases. [D]

10. Open Business Decisions (Client Resolution Required)

ID / Priority	Decision Needed from Owner	Business Impact & Owner Action
O-01 High	Clarify exact product cost assignment method (FIFO vs. Weighted Avg) and confirm whether 2% tax is added to or included in selling price.	Impacts Gross Profit accuracy — Owner confirmation required.
O-02 High	Confirm exact rack selection priority sequence when auto-deducting stock across multiple physical racks.	Impacts POS checkout automation — Owner & Stock Worker input.
O-03 Medium	Confirm overdue credit fee structure (percentage vs. fixed penalty) and formalize list of eligible commercial credit customers.	Impacts Credit Ledger module — Owner decision required.
O-04 Critical	Formally approve Version 1 MVP scope, role permissions matrix, and single-developer delivery timeline.	Impacts project baseline approval — Owner sign-off needed.

11. Acceptance and Validation Plan

System validation will execute automated JUnit test suites for backend APIs and manual acceptance walkthroughs for shop workflows:

Test ID	Proposed Acceptance Scenario & Test Action	Traceability
AT-01	Receive 20 units into Rack A; complete POS sale of 3 units; verify rack balance shows 17 and stock movement log records transaction.	PUR-01, INV-03, POS-04
AT-02	Product SKU has 10 units in Rack A and 5 in Rack B. Sale of 12 units auto-allocates 10 from Rack A and 2 from Rack B without negative stock.	INV-01, INV-05
AT-03	Cashier attempts sale below target margin; system displays margin warning but allows completion per owner configuration.	POS-03, BR-01
AT-04	Issue 12,000 LKR credit sale to approved customer; process 5,000 LKR partial payment; verify outstanding account balance updates to 7,000 LKR.	CRD-02, CRD-03

AT-05	Data-Entry account attempts to alter product price via REST API; Spring Security blocks request with HTTP 403 Forbidden.	AUTH-02, AUTH-03
AT-06	Upload Excel catalog containing valid rows and duplicate SKUs; system displays error preview and awaits user confirmation before database write.	IMP-01, IMP-02
AT-07	Execute JUnit unit test suite during GitHub Actions CI build; verify 100% test pass rate for margin and credit ledger modules.	NFR-TEST-01, NFR-CICD-01

12. Sign-off and Approval Checkpoint

This SRS v1.0 document establishes the formal requirements baseline for the Power Vision Engineering Business Management System. Implementation will proceed according to the single-developer technical stack specified herein upon owner signature.

Approval Checkpoint	Responsible Party	Status / Date / Signature
Business Requirements & Scope Baseline	Power Vision Engineering Owner	Pending Client Review
Technical Stack & Single Dev Baseline	Solo Student Developer	Confirmed (17 October 2026)
Version 1 Role Permissions Matrix	Owner & Development Lead	Pending Client Review
SRS v1.0 Final Sign-Off for Implementation	Owner / Approver	Pending Signature